



Formation Rust

Programmation Systeme Fiable et Performante

De zero a la maitrise des concepts fondamentaux

Module 1

Se familiariser avec Rust

Un peu d'histoire

2006 — Graydon Hoare démarre Rust comme projet personnel chez Mozilla

2009 — Mozilla sponsorise officiellement le projet

2010 — Première annonce publique

2012 — Première version alpha du compilateur (écrit en Rust)

2015 — **Rust 1.0** est publié (15 mai 2015), garantie de stabilité

2021 — Création de la **Rust Foundation** (AWS, Google, Huawei, Microsoft, Mozilla)

2024–2025 — Adoption massive : Linux kernel, Android, Windows, Chromium

*Rust a été élu "langage le plus aimé" sur Stack Overflow pendant **8 années consécutives**.*

Les inspirations de Rust

Concept	Inspire par
Système de types, pattern matching	OCaml, Haskell, ML
Traits et génériques	Haskell (typeclasses)
Gestion mémoire sans GC	C, C++ (RAII)
Macros hygiéniques	Scheme
Concurrence par messages	Erlang
Closures, itérateurs	Ruby, Haskell

Rust emprunte le meilleur de chaque monde pour créer un langage unique.

Motivations pour les fonctionnalités essentielles

Probleme : Le C et le C++ sont performants mais sources de bugs memoire critiques (buffer overflow, use-after-free, data races).

La reponse de Rust :

- **Ownership** : chaque valeur a un unique proprietaire → pas de double-free
- **Borrow Checker** : verification a la compilation des references → pas de dangling pointers
- **Pas de Garbage Collector** : performances previsibles, adaptees a l'embarque
- **Zero-Cost Abstractions** : les abstractions haut-niveau compilent vers du code aussi rapide que le C
- **Thread Safety** : les data races sont impossibles grace au systeme de types

"Si ça compile, ça ne crashe (presque) pas." — Adage de la communauté Rust

Module 2

Installer l'environnement de developpement

Installation de l'environnement

Methode recommandee : `rustup` (gestionnaire de toolchains)

```
# Linux / macOS
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Windows : telecharger rustup-init.exe depuis https://rustup.rs
```

Verification :

```
rustc --version      # Compilateur
cargo --version      # Gestionnaire de paquets / build
rustup --version      # Gestionnaire de toolchains
```

Mise a jour :

```
rustup update
```

Le compilateur et le systeme de build

rustc — Le compilateur Rust (basé sur LLVM)

```
rustc main.rs          # Compile un fichier unique
rustc -O main.rs       # Compile avec optimisations
```

cargo — L'outil tout-en-un (build + deps + tests + docs)

```
cargo new mon_projet    # Cree un nouveau projet
cargo build              # Compile (mode debug)
cargo build --release    # Compile (mode release, optimise)
cargo run                # Compile + execute
cargo check              # Verifie sans generer de binaire (rapide)
```

*En pratique, on utilise **toujours** cargo, rarement rustc directement.*

Le gestionnaire de paquets

Cargo.toml — Le manifeste du projet (equivalent de `pom.xml`, `package.json`)

```
[package]
name = "mon_projet"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { version = "1.0", features = ["derive"] }
tokio = { version = "1", features = ["full"] }
```

crates.io — Le registre officiel de paquets Rust

```
cargo add serde          # Ajoute une dependance
cargo update              # Met a jour les dependances
cargo tree                # Affiche l'arbre de dependances
```

La gestion des tests

Rust integre un framework de tests natif.

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_addition() {
        assert_eq!(2 + 2, 4);
    }

    #[test]
    #[should_panic(expected = "overflow")]
    fn test_overflow() {
        panic!("overflow");
    }
}
```

```
cargo test          # Lance tous les tests
cargo test nom_du_test  # Lance un test specifique
cargo test -- --nocapture  # Affiche les println! pendant les tests
```

La documentation

rustdoc genere une documentation HTML depuis les commentaires.

```
/// Calcule la somme de deux entiers.  
///  
/// # Examples  
///  
/// ```  
/// let resultat = mon_crate::additionner(2, 3);  
/// assert_eq!(resultat, 5);  
/// ```  
pub fn additionner(a: i32, b: i32) -> i32 {  
    a + b  
}
```

```
cargo doc --open      # Genere et ouvre la doc  
cargo test --doc      # Execute les exemples de la doc comme tests
```

Les exemples dans la doc sont testes automatiquement !

Les editeurs et les modes

Editeur	Extension / Plugin
VS Code	<code>rust-analyzer</code> (recommande)
IntelliJ / CLion	Plugin Rust officiel
Neovim	<code>rust-analyzer</code> via LSP

Outils complementaires :

```
rustup component add clippy      # Linter avance
rustup component add rustfmt     # Formateur de code

cargo clippy                     # Analyse statique
cargo fmt                        # Formate tout le projet
```

Module 3

Comprendre les concepts de base

Les conventions de syntaxe

```
// Variables : snake_case
let ma_variable = 42;
let mut compteur = 0;           // Mutable

// Fonctions : snake_case
fn calculer_total(prix: f64, quantite: u32) -> f64 { /* ... */ }

// Types, Structs, Enums : PascalCase
struct MonStruct { /* ... */ }
enum Couleur { Rouge, Vert, Bleu }

// Constantes : SCREAMING_SNAKE_CASE
const MAX_TENTATIVES: u32 = 3;
static VERSION: &str = "1.0.0";

// Modules et crates : snake_case
mod mon_module;
```

*Rust applique ces conventions via **cargo clippy** (warnings si non respectées).*

Les types scalaires

Categorie	Types	Exemples
Entiers signes	i8, i16, i32, i64, i128, isize	let x: i32 = -42;
Entiers non signes	u8, u16, u32, u64, u128, usize	let y: u32 = 42;
Flottants	f32, f64	let pi: f64 = 3.14;

```
let entier = 42;           // i32 par default
let flottant = 3.14;       // f64 par default
let gros = 1_000_000;      // Separateur lisible
let hex = 0xff;            // Hexadecimal
let bin = 0b1010;         // Binaire
let octet: u8 = b'A';     // Byte
```

Les types composes

Tuples — Regroupent des valeurs de types differents

```
let point: (f64, f64) = (3.0, 4.5);  
let (x, y) = point;           // Destructuration  
let premier = point.0;        // Acces par index
```

Tableaux — Taille fixe, meme type

```
let nombres: [i32; 5] = [1, 2, 3, 4, 5];  
let zeros = [0; 10];          // 10 zeros  
let premier = nombres[0];      // Acces par index  
let tranche = &nombres[1..3]; // Slice : [2, 3]
```

Strings — Deux types principaux

```
let s1: &str = "Hello";        // String slice (statique)  
let s2: String = String::from("Hello"); // String (heap, growable)
```


Les expressions

En Rust, **presque tout est une expression** qui retourne une valeur.

```
// Un bloc est une expression
let resultat = {
    let x = 5;
    let y = 10;
    x + y          // Pas de point-virgule = valeur de retour
};
// resultat == 15

// if est une expression
let statut = if resultat > 10 { "grand" } else { "petit" };

// match est une expression
let description = match resultat {
    0 => "zero",
    1..=10 => "petit",
    _ => "grand",
};
```

Attention : ajouter **;** à la fin d'un bloc transforme l'expression en statement (retourne **()**).

Les fonctions

```
// Fonction simple
fn additionner(a: i32, b: i32) -> i32 {
    a + b    // Dernière expression = valeur de retour (pas de return)
}

// Retour explicite (pour retour anticipé)
fn diviser(a: f64, b: f64) -> Option<f64> {
    if b == 0.0 {
        return None;    // Retour anticipé
    }
    Some(a / b)          // Retour implicite
}

// Fonction sans retour (retourne () implicitement)
fn afficher(message: &str) {
    println!("Message : {}", message);
}

// Fonctions génériques
fn plus_grand<T: PartialOrd>(a: T, b: T) -> T {
    if a > b { a } else { b }
}
```

Les types definis par l'utilisateur —

Structs

```
// Struct classique
struct Utilisateur {
    nom: String,
    email: String,
    age: u32,
    actif: bool,
}

// Instanciation
let user = Utilisateur {
    nom: String::from("Alice"),
    email: String::from("alice@example.com"),
    age: 30,
    actif: true,
};

// Tuple struct
struct Point(f64, f64, f64);
let origin = Point(0.0, 0.0, 0.0);

// Unit struct
struct Marqueur;
```

Les types definis par l'utilisateur —

Enums

```
// Enum simple
enum Direction {
    Nord,
    Sud,
    Est,
    Ouest,
}

// Enum avec donnees (type somme / algebraic data type)
enum Forme {
    Cercle { rayon: f64 },
    Rectangle { largeur: f64, hauteur: f64 },
    Triangle(f64, f64, f64),    // Tuple variant
}

// Enums standard essentiels
enum Option<T> {
    Some(T),
    None,
}

enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Conditions et branches

```
// if / else if / else
let temperature = 22;
if temperature > 30 {
    println!("Chaud");
} else if temperature > 15 {
    println!("Agreable");
} else {
    println!("Froid");
}

// match (exhaustif et puissant)
let valeur = 42;
match valeur {
    1 => println!("un"),
    2..=10 => println!("petit"),
    11 | 12 => println!("moyen"),
    n if n > 40 => println!("{}", n),
    _ => println!("autre"),
}

// if let (pour un seul pattern)
let option: Option<i32> = Some(42);
if let Some(v) = option {
    println!("Valeur : {}", v);
}
```

Le mode Panic

panic! — Arrêt immédiat du thread courant (erreur irréversible).

```
fn obtenir_element(v: &[i32], index: usize) -> i32 {  
    if index >= v.len() {  
        panic!("Index {} hors limites (taille: {})", index, v.len());  
    }  
    v[index]  
}
```

Quand utiliser panic vs Result :

```
let fichier = std::fs::read_to_string("config.toml")  
    .expect("Impossible de lire le fichier de configuration");
```

Implementations (impl)

```
struct Rectangle {
    largeur: f64,
    hauteur: f64,
}

impl Rectangle {
    // Methode associee ("constructeur")
    fn new(largeur: f64, hauteur: f64) -> Self {
        Self { largeur, hauteur }
    }

    // Methode d'instance (prend &self)
    fn aire(&self) -> f64 {
        self.largeur * self.hauteur
    }

    // Methode qui modifie l'instance
    fn agrandir(&mut self, facteur: f64) {
        self.largeur *= facteur;
        self.hauteur *= facteur;
    }
}

let mut rect = Rectangle::new(10.0, 5.0);
println!("Aire : {}", rect.aire()); // 50.0
rect.agrandir(2.0);
```

Les boucles

```
// loop (boucle infinie, peut retourner une valeur)
let mut compteur = 0;
let resultat = loop {
    compteur += 1;
    if compteur == 10 {
        break compteur * 2; // break avec valeur
    }
};

// while
let mut n = 0;
while n < 5 {
    println!("{}", n);
    n += 1;
}

// for (le plus idiomatique)
for i in 0..5 {
    println!("{}", i);           // 0, 1, 2, 3, 4
}

for element in [10, 20, 30].iter() {
    println!("{}", element);
}

for (index, val) in "hello".chars().enumerate() {
    println!("{}", index, val);
}
```


Module 4

Ownership et mutabilite

Le systeme d'Ownership

Trois regles fondamentales :

1. Chaque valeur a **un et un seul** proprietaire (owner).
2. Quand le proprietaire sort du scope, la valeur est **droppee** (liberee).
3. Il ne peut y avoir qu'un seul proprietaire a la fois.

```
{  
    let s1 = String::from("hello");    // s1 est proprietaire  
    let s2 = s1;                        // s1 est MOVE vers s2  
    // println!("{}", s1);             // ERREUR : s1 n'est plus valide  
    println!("{}", s2);                // OK  
}    // s2 sort du scope, la memoire est liberee
```

Ce systeme elimine a la compilation :

- les **double-free**
- les **use-after-free**
- les **dangling pointers**

References et mutabilite

```
fn longueur(s: &String) -> usize {    // Emprunt immutable
    s.len()
}

fn ajouter_monde(s: &mut String) {    // Emprunt mutable
    s.push_str(", monde!");
}

fn main() {
    let mut texte = String::from("Bonjour");

    // Plusieurs emprunts immutables : OK
    let r1 = &texte;
    let r2 = &texte;
    println!("{}", et {}, r1, r2);

    // Un seul emprunt mutable a la fois
    let r3 = &mut texte;
    ajouter_monde(r3);
    // let r4 = &mut texte;    // ERREUR : deja emprunte mutablement
}
```

Regle : soit N references $\&T$, soit exactement 1 reference $\&mut T$. Jamais les deux.

Semantique Copy et Move

Move (default pour les types alloues sur le heap) :

```
let s1 = String::from("hello");  
let s2 = s1;           // MOVE : s1 est invalide
```

Copy (pour les types simples, sur la stack) :

```
let x = 42;  
let y = x;             // COPY : x reste valide  
println!("{}", x, y); // OK
```

```
#[derive(Clone, Copy)] // Opt-in pour Copy  
struct Point { x: f64, y: f64 }
```

Le pattern matching approfondi

```
enum Commande {
    Quitter,
    Dire(String),
    Deplacer { x: i32, y: i32 },
    ChangerCouleur(u8, u8, u8),
}

fn traiter(cmd: Commande) {
    match cmd {
        Commande::Quitter => println!("Bye"),
        Commande::Dire(msg) => println!("Message: {}", msg),
        Commande::Deplacer { x, y } => println!("Vers ({}, {})", x, y),
        Commande::ChangerCouleur(r, g, b) => {
            println!("RGB: ({}, {}, {})", r, g, b);
        }
    }
}

// Patterns imbriqués
let point = (3, -5);
match point {
    (0, 0) => println!("Origine"),
    (x, 0) | (0, x) => println!("Sur un axe: {}", x),
    (x, y) if x > 0 && y > 0 => println!("Quadrant I"),
    _ => println!("Autre"),
}
```

Les bases des lifetimes

Les lifetimes garantissent que les references restent valides.

```
// ERREUR : reference vers une valeur droppee
// fn dangling() -> &String {
//     let s = String::from("hello");
//     &s    // s est droppee ici, la reference serait invalide
// }
// Annotation de lifetime explicite
fn le_plus_long<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("longue chaine");
    let resultat;
    {
        let s2 = String::from("xyz");
        resultat = le_plus_long(&s1, &s2);
        println!("{}", resultat); // OK ici
    }
    // println!("{}", resultat); // ERREUR : s2 est droppee
}
```

En general, le compilateur *infere* les lifetimes. On les annote quand il ne peut pas.

Les slices

Les slices sont des **vues** (references) sur une portion contigue de donnees.

```
let texte = String::from("Bonjour le monde");

let mot1: &str = &texte[0..7];      // "Bonjour"
let mot2: &str = &texte[8..10];     // "le"
let fin: &str = &texte[11..];       // "monde"
let tout: &str = &texte[..];        // "Bonjour le monde"

// Slices de tableaux
let nombres = [1, 2, 3, 4, 5];
let milieu: &[i32] = &nombres[1..4]; // [2, 3, 4]

// Utilisation pratique
fn premier_mot(s: &str) -> &str {
    let bytes = s.as_bytes();
    for (i, &b) in bytes.iter().enumerate() {
        if b == b' ' {
            return &s[..i];
        }
    }
    s
}
```

Module 5

Le polymorphisme et les traits

Le polymorphisme simple (Generics)

```
// Fonction generique
fn maximum<T: PartialOrd>(liste: &[T]) -> &T {
    let mut max = &liste[0];
    for item in &liste[1..] {
        if item > max {
            max = item;
        }
    }
    max
}

// Struct generique
struct Paire<T, U> {
    premier: T,
    second: U,
}

impl<T, U> Paire<T, U> {
    fn new(premier: T, second: U) -> Self {
        Self { premier, second }
    }
}

let p = Paire::new(42, "hello");
println!("{}", maximum(&[3, 7, 2, 9, 1])); // 9
```

Definir et implementer des traits

```
// Definition d'un trait
trait Forme {
    fn aire(&self) -> f64;
    fn perimetre(&self) -> f64;

    // Methode par default
    fn description(&self) -> String {
        format!("Aire={:.2}, Perimetre={:.2}", self.aire(), self.perimetre())
    }
}

// Implementation pour un type
struct Cercle { rayon: f64 }

impl Forme for Cercle {
    fn aire(&self) -> f64 {
        std::f64::consts::PI * self.rayon * self.rayon
    }
    fn perimetre(&self) -> f64 {
        2.0 * std::f64::consts::PI * self.rayon
    }
}

let c = Cercle { rayon: 5.0 };
println!("{}", c.description());
```

Le polymorphisme contraint (trait bounds)

```
// Syntaxe classique
fn afficher_aire<T: Forme>(forme: &T) {
    println!("Aire : {:.2}", forme.aire());
}

// Syntaxe where (plus lisible avec plusieurs contraintes)
fn comparer<T, U>(a: &T, b: &U) -> String
where
    T: Forme + std::fmt::Debug,
    U: Forme,
{
    if a.aire() > b.aire() {
        format!("{:?} est plus grand", a)
    } else {
        String::from("b est plus grand ou egal")
    }
}

// Syntaxe impl Trait (en retour)
fn creer_forme(rayon: f64) -> impl Forme {
    Cercle { rayon }
}
```

Les bases des trait objects

```
// Dispatch statique (monomorphisation, zero-cost)
fn afficher_statique(f: &impl Forme) {
    println!("{}", f.description());
}

// Dispatch dynamique (trait object, via vtable)
fn afficher_dynamique(f: &dyn Forme) {
    println!("{}", f.description());
}

// Collection heterogene avec trait objects
fn main() {
    let formes: Vec<Box<dyn Forme>> = vec![
        Box::new(Cercle { rayon: 3.0 }),
        Box::new(Rectangle { largeur: 4.0, hauteur: 5.0 }),
    ];

    for forme in &formes {
        println!("{}", forme.description());
    }
}
```

Module 6

L'ordre superieur

Les traits de fonctions : Fn, FnMut, FnOnce

```
fn appliquer<F: Fn(i32) -> i32>(f: F, x: i32) -> i32 {
    f(x)
}

fn appliquer_mut<F: FnMut()>(mut f: F) {
    f();
    f();
}

fn appliquer_once<F: FnOnce() -> String>(f: F) -> String {
    f()    // Consomme f, ne peut pas etre rappele
}
```

Les closures

```
// Closure simple (inference de types)
let doubler = |x| x * 2;
println!("{}", doubler(5)); // 10

// Closure avec annotation de types
let additionner = |a: i32, b: i32| -> i32 { a + b };

// Capture de l'environnement par reference (&)
let nom = String::from("Alice");
let saluer = || println!("Bonjour, {} !", nom); // Capture &nom
saluer();
println!("{}", nom); // nom est toujours valide

// Capture par reference mutable (&mut)
let mut total = 0;
let mut ajouter = |x: i32| total += x; // Capture &mut total
ajouter(5);
ajouter(10);
drop(ajouter); // Libere l'emprunt
println!("Total : {}", total); // 15
```

Les closures move

```
// Closure move : prend l'ownership des valeurs capturees
let nom = String::from("Alice");
let saluer = move || println!("Bonjour, {} !", nom);
// println!("{}", nom); // ERREUR : nom a ete "moved" dans la closure

saluer();

// Usage principal : passer des donnees a un thread
use std::thread;

let donnees = vec![1, 2, 3];
let handle = thread::spawn(move || {
    // Le thread prend l'ownership de donnees
    println!("Depuis le thread : {:?}", donnees);
});
handle.join().unwrap();
// donnees n'est plus accessible ici

// Retourner une closure
fn creer_incrementeur(pas: i32) -> impl Fn(i32) -> i32 {
    move |x| x + pas
}
```


Module 7

Les collections

Vec — Le tableau dynamique

```
// Creation
let mut v: Vec<i32> = Vec::new();
let v2 = vec![1, 2, 3, 4, 5];

// Ajout / Suppression
v.push(10);
v.push(20);
let dernier = v.pop();    // Some(20)

// Acces
let premier = v[0];        // Panic si hors limites
let safe = v.get(0);        // Option<i32>

// Iteration
for val in &v {
    println!("{}", val);
}

// Methodes utiles
v.extend([30, 40, 50]);
v.retain(|&x| x > 15);        // Garde seulement les > 15
v.sort();
v.dedup();                    // Supprime les doublons consecutifs
let somme: i32 = v.iter().sum();
```

HashMap et BTreeMap

```
use std::collections::{HashMap, BTreeMap};

// HashMap : acces O(1), pas d'ordre
let mut scores: HashMap<String, u32> = HashMap::new();
scores.insert(String::from("Alice"), 100);
scores.insert(String::from("Bob"), 85);

// Acces
if let Some(score) = scores.get("Alice") {
    println!("Alice : {}", score);
}

// Entry API (insérer si absent)
scores.entry(String::from("Charlie")).or_insert(0);
*scores.entry(String::from("Alice")).or_insert(0) += 10;

// BTreeMap : acces O(log n), ordonne par cle
let mut btree: BTreeMap<&str, i32> = BTreeMap::new();
btree.insert("banane", 3);
btree.insert("pomme", 5);
btree.insert("cerise", 8);
// Iteration toujours dans l'ordre alphabetique
for (fruit, quantite) in &btree {
    println!("{}", fruit, quantite);
}
```

Les traits Iterator, Intolterator, Collect

```
// Iterator : methodes chainees
let nombres = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

let resultat: Vec<i32> = nombres.iter()
    .filter(|&&x| x % 2 == 0)      // Garde les pairs
    .map(|&x| x * x)              // Eleve au carre
    .collect();                  // Collecte en Vec
// resultat = [4, 16, 36, 64, 100]

// IntoIterator : conversion automatique en iterateur
for val in &nombres {             // &nombres.into_iter()
    print!("{}", val);
}

// Collect vers differents types
let texte = vec!["Hello", " ", "World"];
let phrase: String = texte.into_iter().collect();

let compteur: HashMap<char, usize> = "abracadabra"
    .chars()
    .fold(HashMap::new(), |mut map, c| {
        *map.entry(c).or_insert(0) += 1;
        map
    });
```

Adaptateurs d'iterateurs essentiels

```
let v = vec![1, 2, 3, 4, 5];

// map, filter, enumerate, zip
let doubles: Vec<_> = v.iter().map(|x| x * 2).collect();
let pairs: Vec<_> = v.iter().filter(|x| *x % 2 == 0).collect();

for (i, val) in v.iter().enumerate() {
    println!("[{}] = {}", i, val);
}

let a = [1, 2, 3];
let b = ["un", "deux", "trois"];
let zipped: Vec<_> = a.iter().zip(b.iter()).collect();

// Reductions
let somme: i32 = v.iter().sum();
let produit: i32 = v.iter().product();
let max = v.iter().max();
let any_pair = v.iter().any(|x| x % 2 == 0);
let all_positifs = v.iter().all(|x| *x > 0);

// take, skip, chain, flatten
let premiers: Vec<_> = v.iter().take(3).collect();
let aplati: Vec<_> = vec![vec![1, 2], vec![3, 4]].into_iter().flatten().collect();
```

Module 8

La concurrence sans peur

Rc et Arc

```
use std::rc::Rc;           // Reference Counted (mono-thread)
use std::sync::Arc;        // Atomic Reference Counted (multi-thread)

// Rc : partage d'ownership dans un seul thread
let a = Rc::new(String::from("partage"));
let b = Rc::clone(&a);     // Incremente le compteur
let c = Rc::clone(&a);
println!("References : {}", Rc::strong_count(&a)); // 3

// Arc : meme chose mais thread-safe
use std::thread;
let data = Arc::new(vec![1, 2, 3]);

let handles: Vec<_> = (0..3).map(|i| {
    let data = Arc::clone(&data);
    thread::spawn(move || {
        println!("Thread {} : {:?}", i, data);
    })
}).collect();

for h in handles {
    h.join().unwrap();
}
```

Rc = mono-thread (pas de surcout atomique). **Arc** = multi-thread (surcout atomique minimal).

Send et Sync

Deux traits auto-implementes qui garantissent la surete en concurrence :

```
// La plupart des types sont Send + Sync
// Rc<T> n'est PAS Send (compteur non atomique)
// Arc<T> EST Send + Sync

// Mutex<T> rend un T accessible de maniere thread-safe
use std::sync::{Arc, Mutex};

let compteur = Arc::new(Mutex::new(0));
let mut handles = vec![];

for _ in 0..10 {
    let compteur = Arc::clone(&compteur);
    handles.push(std::thread::spawn(move || {
        let mut val = compteur.lock().unwrap();
        *val += 1;
    }));
}
for h in handles { h.join().unwrap(); }
println!("Compteur : {}", *compteur.lock().unwrap()); // 10
```


Lancer des threads et passer des messages

```
use std::thread;
use std::sync::mpsc;    // Multi-Producer, Single-Consumer

// Creer un canal
let (tx, rx) = mpsc::channel();

// Cloner l'emetteur pour plusieurs producteurs
let tx2 = tx.clone();

thread::spawn(move || {
    let messages = vec!["bonjour", "depuis", "thread 1"];
    for msg in messages {
        tx.send(msg).unwrap();
        thread::sleep(std::time::Duration::from_millis(100));
    }
});

thread::spawn(move || {
    tx2.send("salut depuis thread 2").unwrap();
});

// Recevoir les messages
for recu in rx {
    println!("Recu : {}", recu);
}
```

"Do not communicate by sharing memory; instead, share memory by communicating." — Go proverb, applicable a Rust aussi.

Mutex et RwLock

```
use std::sync::{Arc, Mutex, RwLock};
use std::thread;

// RwLock : plusieurs lecteurs OU un seul ecrivain
let config = Arc::new(RwLock::new(String::from("v1.0")));

let config_reader = Arc::clone(&config);
let reader = thread::spawn(move || {
    let val = config_reader.read().unwrap();
    println!("Config lue : {}", *val);
});

let config_writer = Arc::clone(&config);
let writer = thread::spawn(move || {
    let mut val = config_writer.write().unwrap();
    *val = String::from("v2.0");
    println!("Config mise a jour");
});

reader.join().unwrap();
writer.join().unwrap();
```

Mutex : acces exclusif. **RwLock** : plusieurs lecteurs simultanement, un ecrivain exclusif.

Recapitulatif

Les piliers de Rust

Ownership — Gestion memoire sans GC

Borrow Checker — Verification statique des references

Traits — Polymorphisme puissant et composable

Fearless Concurrency — Data races impossibles

"Rust : la performance du C, la surete d'un langage manage."

Merci !

Questions ?

Ressources : doc.rust-lang.org | crates.io | play.rust-lang.org